

Aprendizado Profundo

Autoencoders e Variational Autoencoders

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory
PUCRS

maio de 2026

Visão Geral

1. Autoencoders
2. Variational Autoencoders
3. Treinamento e ELBO
4. Arquitetura VAE
5. Reparametrization Trick
6. Panorama

Overview

1. Autoencoders

2. Variational Autoencoders

3. Treinamento e ELBO

4. Arquitetura VAE

5. Reparametrization Trick

6. Panorama

Autoencoders

Um *autoencoder* é uma rede neural treinada na tarefa de **reconstrução da própria entrada**.

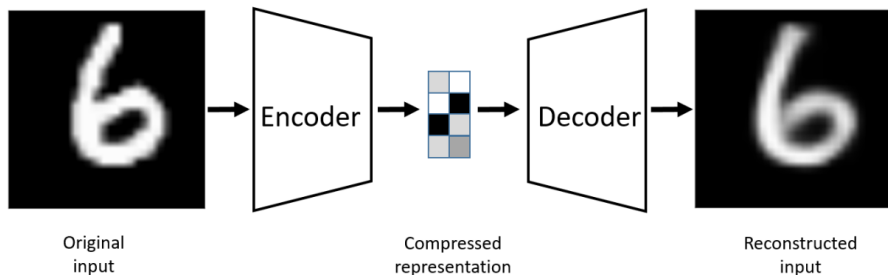
- Uma das primeiras formas de aprendizado **auto-supervisionado** / não supervisionado.
- A ideia é criar uma **representação útil** das instâncias de interesse na etapa intermediária (gargalo).

Objetivo

$$\operatorname{argmin}_{\theta, \phi} J(\mathbf{X}, g(f(\mathbf{X}, \theta), \phi))$$

onde f é o *encoder* e g é o *decoder*.

Autoencoders – Pipeline



- *Encoder* f reduz a entrada a uma representação compacta (o **latente**).
- *Decoder* g tenta reconstruir a entrada original a partir do latente.

Autoencoders – Regularização

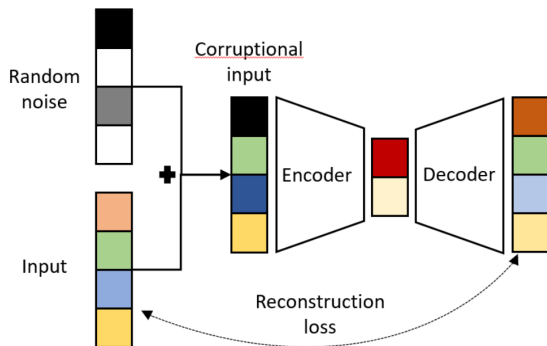
- É importante que o *autoencoder* tenha **regularização** para evitar soluções triviais.
- Se **não houver um gargalo**, o *autoencoder* pode simplesmente copiar a imagem.
- Se houver **unidades suficientes** no vetor latente, é possível indexar as imagens originais (memorização).

Formas comuns de regularização

- **Bottleneck**: dimensão latente $\ll$ dimensão da entrada.
- **Ruído** na entrada (denoising AE).
- **Esparsidade** na ativação do latente.

Denoising Autoencoders¹

- Alguns *autoencoders* são treinados com o objetivo de **remover ruído** das instâncias de entrada.
- Tipicamente o *noise* segue uma:
 - distribuição **normal** (ruído aditivo);
 - distribuição de **Bernoulli** (ruído multiplicativo que invalida algumas *features*).
- A reconstrução é avaliada contra a entrada original (sem ruído).



¹Pascal Vincent et al. "Extracting and Composing Robust Features with Denoising Autoencoders". Em: *Proceedings of the 25th International Conference on Machine Learning*. 2008, pp. 1096–1103.

Overview

1. Autoencoders
- 2. Variational Autoencoders**
3. Treinamento e ELBO
4. Arquitetura VAE
5. Reparametrization Trick
6. Panorama

Variational Autoencoders²

- São modelos geradores **probabilísticos** cujo objetivo é aprender uma distribuição $p(x)$ sobre os dados.
- É um **modelo não linear de variável latente**.
 - Depois do treinamento podemos usar apenas o decoder (amostragem) ou apenas o encoder (inferência).

Panorama

O VAE é a **fusão** de duas ideias:

- *Autoencoder*: arquitetura encoder/decoder.
- **Inferência variacional**: aproximar a verossimilhança intratável via um limite inferior (ELBO).

²Diederik P. Kingma e Max Welling. “Auto-Encoding Variational Bayes”. Em: International Conference on Learning Representations. 2014.

Modelos de variáveis latentes

- Abordagem **indireta** para modelar uma distribuição $p(x)$ sobre uma variável multidimensional x .
- Marginalização de uma probabilidade conjunta $p(x, z)$:

$$p(x) = \int p(x, z) dz = \int p(x | z) p(z) dz$$

Motivação

Distribuições relativamente simples $p(x | z)$ e $p(z)$ podem, por marginalização, definir distribuições $p(x)$ **complexas**.

Exemplo: Mistura de Gaussianas 1D

- z é uma variável latente **discreta**, com distribuição a priori categórica: $p(z = n) = \lambda_n$.
- A verossimilhança segue uma normal: $p(x | z = n) = \mathcal{N}(\mu_n, \sigma_n^2)$.
- A distribuição marginal:

$$p(x) = \sum_n \lambda_n \mathcal{N}(\mu_n, \sigma_n^2).$$

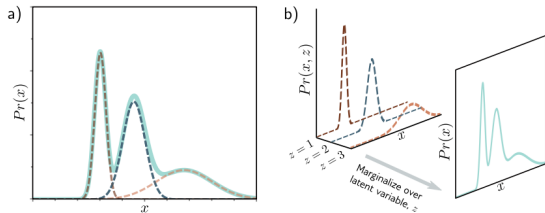


Figure 17.1 Mixture of Gaussians (MoG). a) The MoG describes a complex probability distribution (cyan curve) as a weighted sum of Gaussian components (dashed curves). b) This sum is the marginalization of the joint density $Pr(x, z)$ between the continuous observed data x and a discrete latent variable z .

Modelos não lineares de variáveis latentes

Tanto z quanto x são **contínuos** e **multivariados**.

- *Prior* normal padrão: $p(z) = \mathcal{N}_z(0, \mathbf{I})$.
- Verossimilhança gaussiana com média não linear $f(z, \boldsymbol{\theta})$ e covariância esférica:

$$p(x | z, \boldsymbol{\theta}) = \mathcal{N}_x\left(f(z, \boldsymbol{\theta}), \sigma^2 \mathbf{I}\right).$$

- A distribuição de interesse é obtida por marginalização:

$$p(x | \boldsymbol{\theta}) = \int \mathcal{N}_x\left(f(z, \boldsymbol{\theta}), \sigma^2 \mathbf{I}\right) \mathcal{N}_z(0, \mathbf{I}) dz.$$

Gerando novas instâncias x^*

Usamos **amostragem ancestral**:

1. Amostrar um latente z^* de $p(z)$ (*prior*).
2. Passar z^* pela rede $f(z^*, \theta)$ para obter a média da verossimilhança $p(x | z^*, \theta)$.
3. Amostrar x^* de $p(x | z^*, \theta)$.

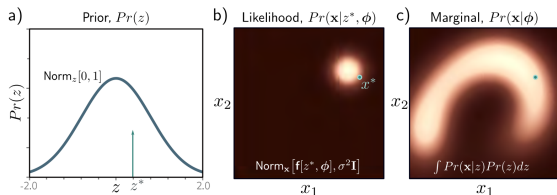


Figure 17.3 Generation from nonlinear latent variable model. a) We draw a sample z^* from the prior probability $Pr(z)$ over the latent variable. b) A sample $\mathbf{x}^*$ is then drawn from $Pr(\mathbf{x}|z^*, \phi)$. This is a spherical Gaussian with a mean that is a nonlinear function $\mathbf{f}[\bullet, \phi]$ of z^* and a fixed variance $\sigma^2 \mathbf{I}$. c) If we repeat this process many times, we recover the density $Pr(\mathbf{x}|\phi)$.

Overview

1. Autoencoders
2. Variational Autoencoders
- 3. Treinamento e ELBO**
4. Arquitetura VAE
5. Reparametrization Trick
6. Panorama

Treinamento – O problema

Maximizar a log-verossimilhança direta é **inviável**: não existe expressão fechada para a integral.

$$\hat{\boldsymbol{\theta}} = \operatorname{argmax}_{\boldsymbol{\theta}} \left[\sum_{i=1}^N \int \mathcal{N}_{x^{(i)}} \left(f(z, \boldsymbol{\theta}), \sigma^2 \mathbf{I} \right) \mathcal{N}_z(0, \mathbf{I}) dz \right]$$

Estratégia

Não otimizamos a log-verossimilhança diretamente. Em vez disso, construímos e maximizamos um **limite inferior** para ela.

Treinamento – Limite inferior

- Como não conseguimos maximizar diretamente o $\log p(x \mid \theta)$, criamos uma expressão que **garantidamente** é menor ou igual à log-verossimilhança.
- Essa será nossa **função objetivo** otimizável.
- Para implementá-la, adicionamos uma **segunda rede** com parâmetros treináveis ϕ distintos de θ :
 - ela vai aproximar a distribuição posterior $p(z \mid x)$ via uma distribuição $q(z \mid x, \phi)$.

Desigualdade de Jensen

Para qualquer função **côncava** $g(\cdot)$:

$$g(\mathbb{E}[x]) \geq \mathbb{E}[g(x)]$$

Ao lado, o caso da função $\log$:

$$\log(\mathbb{E}[y]) \geq \mathbb{E}[\log y]$$

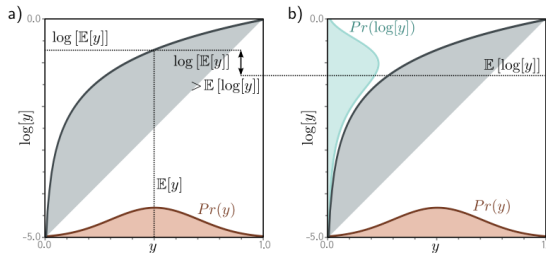


Figure 17.5 Jensen's inequality (continuous case). For a concave function, computing the expectation of a distribution $Pr(y)$ and passing it through the function gives a result greater than or equal to transforming the variable y by the function and then computing the expectation of the new variable. In the case of the logarithm, we have $\log[\mathbb{E}[y]] \geq \mathbb{E}[\log[y]]$. The left-hand side of the figure corresponds to the left-hand side of this inequality and the right-hand side of the figure to the right-hand side. One way of thinking about this is to consider that we are taking a convex combination of the points in the orange distribution defined over $y \in [0, 1]$. By the logic of figure 17.4, this must lie under the curve. Alternatively, we can think about the concave function as compressing the high values of y relative to the low values, so the expected value is lower when we pass y through the function first.

Limite Inferior – Derivação

Usando Jensen para derivar o limite inferior da log-verossimilhança:

$$\begin{aligned}\log[p(x \mid \boldsymbol{\theta})] &= \log \left[\int p(x, z \mid \boldsymbol{\theta}) dz \right] \\ &= \log \left[\int q(z) \frac{p(x, z \mid \boldsymbol{\theta})}{q(z)} dz \right] \\ &\geq \int q(z) \log \left(\frac{p(x, z \mid \boldsymbol{\theta})}{q(z)} \right) dz \quad (\text{Jensen})\end{aligned}$$

ELBO

Esse termo é conhecido como *evidence lower bound* (ELBO). Na prática, a distribuição auxiliar também é parametrizada: $q(z \mid \phi)$.

ELBO – Reorganizando

$$\text{ELBO}[\boldsymbol{\theta}, \phi] = \int q(z | \phi) \log \left(\frac{p(x, z | \boldsymbol{\theta})}{q(z | \phi)} \right) dz$$

Aplicando $p(x, z) = p(x | z) p(z)$ e separando a razão:

$$\begin{aligned} &= \int q(z | \phi) \log \left(\frac{p(x | z, \boldsymbol{\theta}) p(z)}{q(z | \phi)} \right) dz \\ &= \int q(z | \phi) \log p(x | z, \boldsymbol{\theta}) dz - D_{KL}[q(z | \phi) \| p(z)] \end{aligned}$$

ELBO – Interpretação

$$\text{ELBO}[\boldsymbol{\theta}, \phi] = \underbrace{\int q(z | \phi) \log p(x | z, \boldsymbol{\theta}) dz}_{\text{Reconstruction Loss}} - \underbrace{D_{KL}[q(z | \phi) \| p(z)]}_{\text{Regularização do latente}}$$

- **Primeiro termo:** avalia a concordância média entre a variável latente e os dados. Também chamado de *reconstruction loss*.
- **Segundo termo:** avalia quão perto a distribuição auxiliar $q(z | \phi)$ está do *prior* $p(z)$.

VAE – Condicionando em x

Adicionamos mais um **condicionante em x** para termos um limite inferior mais justo:

$$\text{ELBO}[\boldsymbol{\theta}, \phi] = \int q(z \mid x, \phi) \log p(x \mid z, \boldsymbol{\theta}) dz - D_{KL}[q(z \mid x, \phi) \parallel p(z)]$$

- O *encoder* agora depende de x : produz uma distribuição por exemplo.
- Com ambas as distribuições gaussianas, a KL tem **forma fechada** – vamos derivar nos próximos dois slides.

KL entre gaussianas (1/2) – Partindo da definição

A KL entre duas distribuições quaisquer é:

$$D_{KL}[q \parallel p] = \int q(z) \log\left(\frac{q(z)}{p(z)}\right) dz = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(z)]$$

KL entre gaussianas (1/2) – Partindo da definição

A KL entre duas distribuições quaisquer é:

$$D_{KL}[q \parallel p] = \int q(z) \log\left(\frac{q(z)}{p(z)}\right) dz = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(z)]$$

Para $q(z|x, \phi) = \mathcal{N}(\mu, \Sigma)$ e $p(z) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, o log de cada densidade gaussiana é uma forma quadrática:

$$\log p(z) = -\frac{D_z}{2} \log(2\pi) - \frac{1}{2} z^\top z$$

$$\log q(z) = -\frac{D_z}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (z - \mu)^\top \Sigma^{-1} (z - \mu)$$

KL entre gaussianas (2/2) – Forma fechada

Tomando esperanças sob q (usando $\mathbb{E}_q[z] = \mu$ e $\mathbb{E}_q[zz^\top] = \Sigma + \mu\mu^\top$):

$$\mathbb{E}_q\left[-\frac{1}{2}z^\top z\right] = -\frac{1}{2}\left(\text{Tr}[\Sigma] + \mu^\top \mu\right)$$

$$\mathbb{E}_q\left[-\frac{1}{2}(z - \mu)^\top \Sigma^{-1}(z - \mu)\right] = -\frac{1}{2} D_z$$

Substituindo na definição da KL:

$$D_{KL}[\mathcal{N}(\mu, \Sigma) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})] = \frac{1}{2} \left(\underbrace{\text{Tr}[\Sigma]}_{\text{dispersão}} + \underbrace{\mu^\top \mu}_{\text{desvio da origem}} - \underbrace{D_z}_{\text{norm}} - \underbrace{\log \det \Sigma}_{\text{volume}} \right)$$

Exemplo Numérico: Calculando o Termo KL

Imagine um VAE treinado em MNIST com latente 2D. Para a imagem de um “7” específico, o encoder produz uma distribuição $q(z | x, \phi) = \mathcal{N}(\mu, \Sigma)$ sobre o latente, compacta o suficiente para “codificar 7”, mas com alguma incerteza para suavizar o espaço. O termo KL mede quão longe essa distribuição está do prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

Encoder produz $\mu = (0,5, -0,8)$ e $\Sigma = \text{diag}(0,25, 1,5)$ (latente 2D, $D_z = 2$).

Calculando cada termo:

$$\text{Tr}[\Sigma] = 0,25 + 1,5 = 1,75$$

$$\mu^\top \mu = 0,25 + 0,64 = 0,89$$

$$\log \det \Sigma = \log(0,25) + \log(1,5)$$

$$\approx -1,386 + 0,405$$

$$= -0,981$$

$$D_z = 2$$

Substituindo

$$\begin{aligned} D_{KL} &= \frac{1}{2} (1,75 + 0,89 - 2 + 0,981) \\ &= \frac{1}{2} (1,621) \approx \mathbf{0,81} \end{aligned}$$

Os dois termos que dominam:

$\mu^\top \mu = 0,89$ (desvio da origem) e

$-\log \det \Sigma = +0,98$ (variância pequena $\sigma^2 = 0,25$ é penalizada para evitar colapso).

VAE – Aproximação por *Monte Carlo*

O primeiro termo do ELBO ainda é uma integral intratável. Porém, como é uma **esperança**, podemos aproximá-lo por *Monte Carlo*:

$$\int q(z \mid x, \phi) \log p(x \mid z, \boldsymbol{\theta}) dz \approx \log p(x \mid z^*, \boldsymbol{\theta}), \quad z^* \sim q(z \mid x, \phi)$$

ELBO aproximado

$$\text{ELBO}[\boldsymbol{\theta}, \phi] \approx \log p(x \mid z^*, \boldsymbol{\theta}) - D_{KL}[q(z \mid x, \phi) \parallel p(z)]$$

Overview

1. Autoencoders
2. Variational Autoencoders
3. Treinamento e ELBO
- 4. Arquitetura VAE**
5. Reparametrization Trick
6. Panorama

Arquitectura VAE

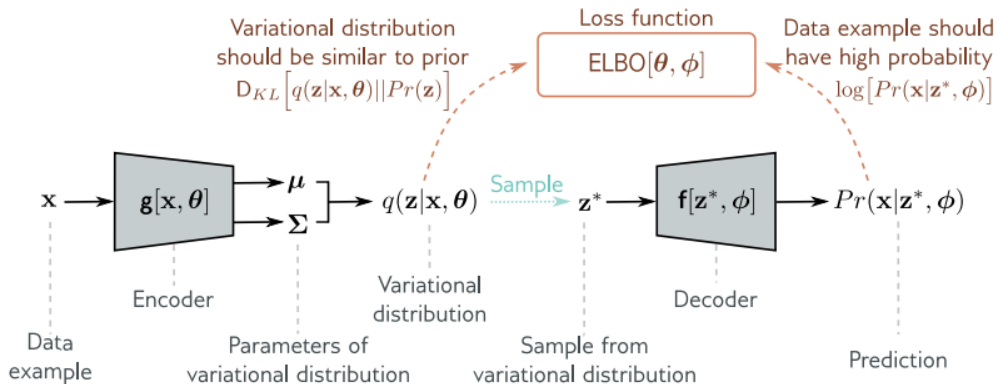


Figure 17.9 Variational autoencoder. The encoder $\mathbf{g}[\mathbf{x}, \theta]$ takes a training example $\mathbf{x}$ and predicts the parameters μ, Σ of the variational distribution $q(\mathbf{z}|\mathbf{x}, \theta)$. We sample from this distribution and then use the decoder $\mathbf{f}[\mathbf{z}, \phi]$ to predict the data $\mathbf{x}$. The loss function is the negative ELBO, which depends on how accurate this prediction is and how similar the variational distribution $q(\mathbf{z}|\mathbf{x}, \theta)$ is to the prior $Pr(\mathbf{z})$ (equation 17.21).

Overview

1. Autoencoders
2. Variational Autoencoders
3. Treinamento e ELBO
4. Arquitetura VAE
- 5. Reparametrization Trick**
6. Panorama

Reparametrization Trick

- Como temos um **termo estocástico** no meio da rede neural (a amostragem $z^* \sim q(z \mid x, \phi)$), é inviável computar *backpropagation* diretamente.
- **Solução:** reparametrizar a saída do *encoder* de modo a tirar a amostragem do caminho do gradiente:

$$z^* = \mu + \Sigma^{1/2} \epsilon^*, \quad \epsilon^* \sim \mathcal{N}(0, \mathbf{I})$$

Ideia chave

O ruído ϵ^* é amostrado fora da rede; μ e Σ permanecem no caminho diferenciável.

Reparametrization Trick – Diagrama

Figure 17.10 The VAE updates both factors that determine the lower bound at each iteration. Both the parameters ϕ of the decoder and the parameters θ of the encoder are manipulated to increase this lower bound.

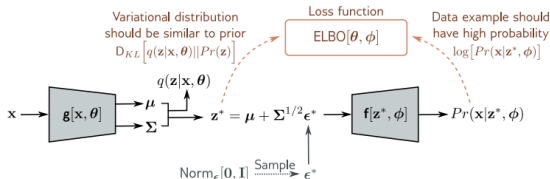
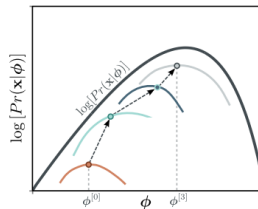


Figure 17.11 Reparameterization trick. With the original architecture (figure 17.9), we cannot easily backpropagate through the sampling step. The reparameterization trick removes the sampling step from the main pipeline; we draw from a standard normal and combine this with the predicted mean and covariance to get a sample from the variational distribution.

Prince (2023), Fig. 17.10 e 17.11.

Overview

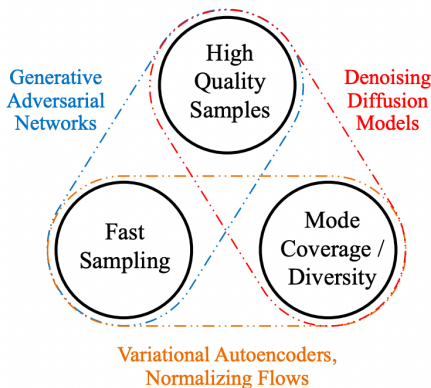
1. Autoencoders
2. Variational Autoencoders
3. Treinamento e ELBO
4. Arquitetura VAE
5. Reparametrization Trick
- 6. Panorama**

Onde VAEs ficam no trilema?

- **Amostragem rápida:** ✓ (1 forward do decoder).
- **Cobertura / diversidade:** ✓ (maximização do ELBO cobre toda a distribuição).
- **Qualidade visual:** × (imagens tipicamente borradas; o limite inferior não é apertado).

Posicionamento

VAEs ocupam o canto velocidade + cobertura do trilema, ao lado dos *Normalizing Flows*. Para qualidade, combine VAEs com modelos difusores – base da arquitetura *Latent Diffusion*.



Resumo

- *Autoencoders*: encoder/decoder reconstruindo a entrada; latente útil para representação.
- *Denoising AE*: entrada corrompida; rede aprende a remover ruído.
- *VAE*: *autoencoder* probabilístico, o latente é uma distribuição.
- Treinado maximizando o **ELBO**, limite inferior da log-verossimilhança.
- ELBO = reconstrução + KL entre $q(z | x, \phi)$ e o *prior* $p(z)$.
- *Reparametrization trick* permite backprop através da amostragem.

Leituras Recomendadas

- Capítulo 17: Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023
- Diederik P. Kingma e Max Welling. “Auto-Encoding Variational Bayes”. Em: International Conference on Learning Representations. 2014
- Pascal Vincent et al. “Extracting and Composing Robust Features with Denoising Autoencoders”. Em: Proceedings of the 25th International Conference on Machine Learning. 2008, pp. 1096–1103
- Irina Higgins et al. “ β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework”. Em: International Conference on Learning Representations. 2017

Referências I

- [1] Irina Higgins et al. “ β -VAE: Learning Basic Visual Concepts with a Constrained Variational Framework”. Em: International Conference on Learning Representations. 2017.
- [2] Diederik P. Kingma e Max Welling. “Auto-Encoding Variational Bayes”. Em: International Conference on Learning Representations. 2014.
- [3] Simon J. D. Prince. Understanding Deep Learning. MIT Press, 2023.
- [4] Pascal Vincent et al. “Extracting and Composing Robust Features with Denoising Autoencoders”. Em: Proceedings of the 25th International Conference on Machine Learning. 2008, pp. 1096–1103.